

Lecture 3

Stochastic gradient descent

Historical Roots of Stochastic Gradient Descent

- **Robbins and Monro (1951)** introduced stochastic approximation: iterative root-finding from noisy observations. Their original method was not presented as gradient descent.
- Modern **stochastic gradient descent** is a stochastic-approximation algorithm: each update uses a sample or mini-batch estimate of the full gradient.
- **Kiefer and Wolfowitz (1952)** extended stochastic approximation to optimization using noisy function measurements.

References: Robbins & Monro, Annals of Mathematical Statistics 22(3), 1951; Bottou & Bousquet, NeurIPS 20 (NIPS 2007), pp. 161–168.

Stochastic gradient descent

Suppose that we want to minimize an objective function that is written as a sum of differentiable functions.

$$Q(w) = \sum_{i=1}^n Q_i(w)$$

Each term Q_i is usually associated with the i -th data point.

Standard gradient descent (batch gradient descent):

$$w = w - \eta \nabla Q(w) = w - \eta \sum_{i=1}^n \nabla Q_i(w)$$

where η is the learning rate (step size).

Stochastic gradient descent

The gradient of $Q(w)$ is approximated by a gradient at a single example:

$$w = w - \eta \nabla Q_i(w).$$

This update needs to be done for each training example.

Several passes might be necessary over the training set until the algorithm converges.

η might be adaptive.

Stochastic gradient descent

- Choose an initial value for w and η .
- Repeat until converged
 - Randomly shuffle data points in the training set.
 - For $i = 1, 2, \dots, n$, do:
 - $w = w - \eta \nabla Q_i(w)$.

Why Stochastic Gradient Descent (SGD) Works

- Problem Statement:
- We want to minimize a loss function $Q(\mathbf{w})$ where $\mathbf{w}$ is the parameter vector.
- The loss function is assumed to be differentiable.
- Show that Stochastic Gradient Descent (SGD) converges to a local minimum of $Q(\mathbf{w})$.

Why Stochastic Gradient Descent (SGD) Works

- Initialization: Start with an initial guess w_0
- Update Rule:

$$w_{\{t+1\}} = w_t - \rho \nabla Q_{i(w_t)}.$$

- Expectation of the Gradient:

$$E[\nabla Q_{i(w_t)}] = \nabla Q(w_t).$$

- Convergence: If ρ is sufficiently small, then each update moves w_t closer to the minimum of $Q(w)$.

Why Stochastic Gradient Descent (SGD) Works

- Expectation of Update Rule:

$$E[w_{\{t+1\}}] = E[w_t] - \rho E[\nabla Q_{i(w_t)}]$$

- Substitute Expectation of Gradient:

$$E[w_{\{t+1\}}] = E[w_t] - \rho \nabla Q(w_t).$$

- Convergence: This equation implies that the expected value of w moves in the direction of the negative gradient, thus moving towards the minimum of $Q(w)$

Why Stochastic Gradient Descent (SGD) Works

- The proof shows that the expected update of w using SGD is in the direction of the true gradient of the loss function $Q(w)$
- Therefore, SGD works in practice to find a local minimum of the loss function, provided the learning rate ρ is appropriately chosen.

Mini-batches

Batch gradient decent uses all n data points in each iteration.

Stochastic gradient decent uses 1 data point in each iteration.

Mini-batch gradient decent uses b data points in each iteration.

b is a parameter called Mini-batch size.

Mini-batches

- Choose an initial value for w and η .
- Say $b = 10$
- Repeat until converged
 - Randomly shuffle data points in the training set.
 - For $i = 1, 11, 21, \dots, n - 9$, do:
 - $w = w - \eta \sum_{k=i}^{i+9} \nabla Q_i(w)$.

Tuning η

If η is too high, the algorithm diverges.

If η is too low, makes the algorithm slow to converge.

A common practice is to make η_t a decreasing function of the iteration number t . e.g. $\eta_t = \frac{\text{constant1}}{t + \text{constant2}}$

The first iterations cause large changes in the w , while the later ones do only fine-tuning.

Momentum

- Momentum is a technique used in optimization to accelerate convergence.
- Inspired by physical momentum, it helps in navigating the optimization landscape.

Mathematical Formulation

- Standard GD Update:

$$w_{\{t+1\}} = w_t - \rho \nabla Q(w_t)$$

- Momentum Update:

$$v_{\{t+1\}} = \beta v_t + (1 - \beta) \nabla Q(w_t)$$

$$w_{\{t+1\}} = w_t - \rho v_{\{t+1\}}$$

- β : Momentum coefficient (0.9 to 0.99)
- v_t : Velocity term, a running average of gradients

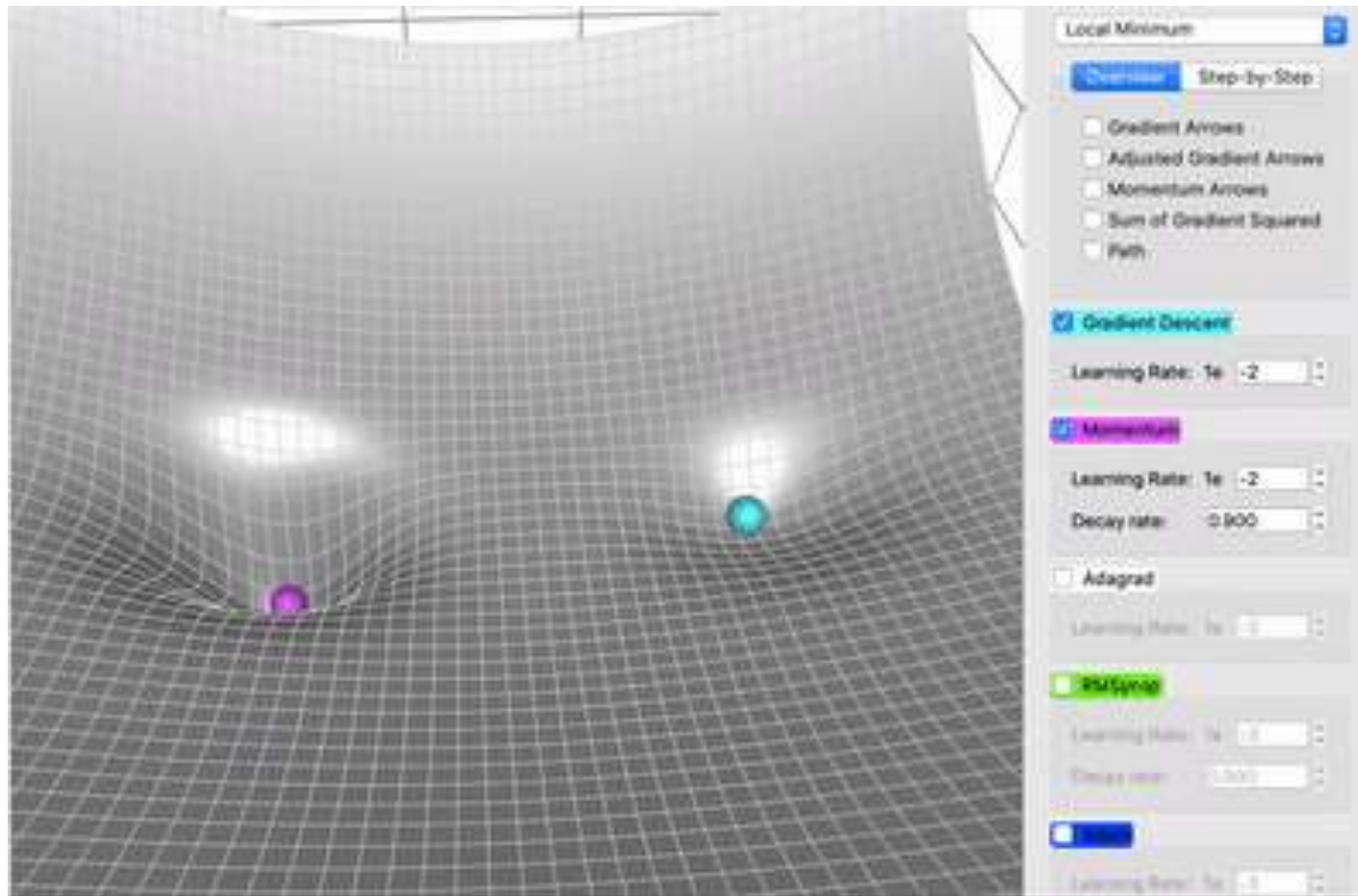
Understanding the Velocity Term

- The velocity term v_t is a running average of past gradients.
- It accumulates information from past updates to inform the next step.
- Mathematical Update:

$$v_{\{t+1\}} = \beta v_t + (1 - \beta) \nabla Q(w_t)$$

Role in Optimization:

- Smoothing out updates
- Accelerating through flat regions
- Providing stability



Momentum (magenta) vs. Gradient Descent (cyan) on a surface with a global minimum (the left well) and local minimum (the right well)

Optimization Algorithms in Deep Learning

- New optimization algorithms, such as **AdamW**, **AMSGrad**, **QHAdam**, **YellowFin**, **Demon Adam**, **Momentum**, **Aggmo QHM**, and **LAMB**, have been proposed to address issues like convergence and generalization.
- **SGD with momentum** is slower to converge but generally performs better in terms of generalization, especially in large-scale and high-dimensional settings.
- The field of optimization is evolving rapidly, with ongoing research to find the best balance between convergence speed and generalization.

The 2026 Optimizer Landscape

ESTABLISHED BASELINES : AdamW for transformers; SGD with momentum for many vision and fine-tuning workloads.

MEMORY AND SCALE : Adafactor and low-memory Adam implementations reduce optimizer-state cost; LAMB and LARS remain useful for very large batches.

EMERGING DIRECTIONS : Muon and NorMuon use matrix-structured updates; SOAP and Shampoo use preconditioning; Schedule-Free methods reduce dependence on a predefined decay schedule; Sophia uses lightweight curvature estimates.

Popular Optimization Algorithms in Deep Learning (2026)

ADAMW : The de-facto baseline for LLMs, transformers, diffusion, and multimodal models; it decouples weight decay from the gradient update.

SGD WITH MOMENTUM : Still a strong choice for CNNs and some vision workloads; often competitive when schedules and augmentation are tuned.

ADAFCTOR : Adam-like adaptation with factored second-moment state; useful when optimizer memory is a binding constraint.

Popular Optimization Algorithms in Deep Learning (2026)

MUON : Orthogonalizes matrix-shaped updates; promising LLM efficiency results, but requires careful parameter grouping and scaling.

SOAP / SHAMPOO : Structured preconditioning can reduce training steps; added matrix operations, memory, and distributed complexity must be justified.

SCHEDULE-FREE ADAMW : Removes the need to specify a conventional decay schedule; useful when the stopping horizon is uncertain.

SOPHIA / LAMB / LARS : Specialized alternatives for curvature-aware LLM experiments or very-large-batch training; not universal defaults.

Rule of Thumb for Choosing an Optimizer

TRANSFORMERS AND FOUNDATION MODELS : Start with AdamW and a proven warmup/decay recipe. Treat it as the baseline to beat.

VISION MODELS : Compare SGD with momentum against AdamW; architecture and augmentation can matter more than optimizer family.

MEMORY-CONSTRAINED TRAINING : Try Adafactor or a validated low-memory Adam implementation; measure convergence and wall-clock cost.

Rule of Thumb for Specialized Optimizers

LARGE-BATCH DISTRIBUTED TRAINING : Consider LAMB or LARS when scaling batch size creates tuning difficulty.

EXPERIMENTAL EFFICIENCY : Benchmark Muon or SOAP when step efficiency matters and the implementation is mature for your model and hardware.

UNKNOWN TRAINING HORIZON : Schedule-Free AdamW can reduce scheduler tuning. Compare final quality, stability, memory, communication, and total time—not steps alone.

Summary — Choosing the Right Optimization Method

USE ADAMW as the default baseline for transformers and most foundation-model training.

USE SGD WITH MOMENTUM when it is established for the architecture, especially in vision; use Adafactor when optimizer-state memory is limiting.

TREAT MUON, SOAP / SHAMPOO, SCHEDULE-FREE ADAMW, AND SOPHIA as promising or specialized options—not universal replacements.

CHOOSE BY final quality, stability, memory, communication, and wall-clock cost. Tune the complete training recipe, then compare fairly.

Challenges of Second-Order Optimization in Deep Learning

- **Computational Complexity:**
 - Requires Hessian matrix computation: $O(n^2)$ complexity.
 - Matrix inversion: $O(n^3)$ complexity.
- **Memory Requirements:**
 - Hessian storage: $O(n^2)$ memory.
- **Non-Convexity Issues:**
 - Risk of getting stuck at saddle points or sharp local minima, which can lead to poor generalization.

Advantages of First-Order Optimization Methods

- **Lower Computational Cost:**
 - Only requires gradient computation: $O(n)$ complexity.
 - No need to compute or store the Hessian matrix.
- **Better Generalization through Flatter Minima:**
 - First-order methods like **SGD** introduce noise that helps avoid sharp minima.
 - Tend to find **flatter minima**, which are associated with better generalization and robustness.
- **Implicit Regularization Effect:**
 - The stochastic nature of SGD acts as a form of regularization, reducing the likelihood of overfitting to training data.

Why Second-Order and Preconditioned Methods Are Hard

EXACT CURVATURE IS INFEASIBLE AT MODERN SCALE — A dense Hessian requires quadratic storage and expensive matrix operations.

APPROXIMATIONS TRADE ACCURACY FOR COST — Diagonal, block, Kronecker, and matrix preconditioners add state, hyperparameters, and numerical issues.

SYSTEMS OVERHEAD CAN DOMINATE — Matrix roots, eigendecompositions, and synchronization may erase step-count gains.

THE RESULT IS WORKLOAD-SPECIFIC — Shampoo, SOAP, and Sophia can help, but are not automatically superior to a tuned first-order baseline.

Why First-Order Methods Still Dominate

SIMPLE AND SCALABLE — Gradients map efficiently to accelerators, data parallelism, and sharded training.

MATURE AND PREDICTABLE — AdamW and SGD have strong implementations, known failure modes, and reusable tuning recipes.

COMPETITIVE END-TO-END — Schedule, clipping, weight decay, precision, and batch size often matter as much as the optimizer name.

FLEXIBLE TRADE-OFFS — AdamW favors robustness; SGD favors simplicity; Adafactor reduces state memory.

Summary - Choosing the Right Optimization Method

- **First-Order Methods (e.g., SGD, Adam):**
 - Preferred for deep learning due to lower complexity, memory efficiency, and better generalization.
 - Robust in navigating non-convex landscapes and avoiding poor local minima.
- **Second-Order Methods:**
 - Less practical for deep learning due to high computational and memory costs.
 - More prone to converge to sharp, narrow minima that do not generalize well.